Aula 3 - Máquinas Virtuais e Containers (DevOps)

Desafio 2: Apresentação em equipe usando Docker ou Kubernetes

Esta aula e desafio visam explorar o universo das Máquinas Virtuais (VMs) e Containers, ferramentas essenciais no cenário DevOps moderno. Através de uma abordagem prática, vocês terão a oportunidade de aplicar os conceitos aprendidos, seja com Docker ou Kubernetes, para construir e apresentar uma solução em equipe.

Made with GAMMA

CONCEITO FUNDAMENTAL

O que são Máquinas Virtuais (VMs)?

Máquinas Virtuais, ou VMs, são emulações de um sistema de computador completo. Elas fornecem a funcionalidade de um computador físico, executando um sistema operacional convidado dentro de outro sistema operacional hospedeiro, através de um software chamado hypervisor.

Emulam hardware completo para rodar sistemas operacionais isolados

Cada VM possui seu próprio CPU virtual, memória, disco e interfaces de rede, permitindo que ela funcione como uma máquina física independente.

Cada VM tem seu próprio sistema operacional, recursos dedicados e isolamento total

Isso significa que falhas em uma VM não afetam as outras, garantindo segurança e estabilidade. Os recursos são pré-alocados, o que pode levar a um desperdício se a VM não utilizar toda a capacidade.

Uso comum para ambientes que precisam simular diferentes hardwares ou SOs

Por exemplo, testar aplicações em múltiplas versões de Windows e Linux simultaneamente, ou consolidar servidores físicos para otimizar o uso de hardware.

Made with GAMMA

EVOLUÇÃO DA VIRTUALIZAÇÃO

O que são Containers?

Containers representam uma abordagem mais leve e eficiente para o isolamento de aplicações. Ao invés de virtualizar o hardware, eles virtualizam o sistema operacional, permitindo que múltiplas aplicações compartilhem o mesmo kernel do SO hospedeiro, mas permaneçam isoladas umas das outras.

Ambientes leves que compartilham o kernel do sistema operacional hospedeiro

Essa característica confere aos containers uma pegada menor e um desempenho mais próximo ao do hardware nativo, comparado às VMs.

Empacotam aplicações com todas as dependências

Isso inclui código, tempo de execução, bibliotecas de sistema, ferramentas de sistema e configurações. Essa portabilidade garante que a aplicação funcione da mesma forma em qualquer ambiente.

Inicialização rápida e uso eficiente de recursos

Os containers iniciam em questão de segundos e consomem significativamente menos recursos de CPU e memória, tornando-os ideais para microsserviços e desenvolvimento ágil.

Made with GAMMA

COMPARAÇÃO E ESCOLHA

Containers vs Máquinas Virtuais: Diferenças Cruciais

Entender as distinções entre VMs e containers é fundamental para escolher a tecnologia certa para cada caso de uso. Enquanto as VMs oferecem isolamento completo de hardware, os containers priorizam a eficiência e a portabilidade através do compartilhamento do kernel.

Máquinas Virtuais (VMs)

- Pesado e com maior consumo de recursos
- Isolam hardware e sistemas operacionais completos
- Tempo de inicialização medido em minutos
- Podem rodar múltiplos sistemas operacionais distintos
- Ótimo para ambientes heterogêneos e legado

Containers

- Leves e com uso eficiente de recursos
- Compartilham o kernel do SO hospedeiro
- Tempo de inicialização medido em segundos
- Foco principal em empacotar e isolar aplicações
- Ideal para microsserviços e desenvolvimento moderno

Um exemplo prático demonstra bem a diferença: Para uma aplicação que requer um servidor web, um banco de dados e um cache, você precisaria de três VMs separadas, cada uma com seu próprio SO e recursos. Com containers, você teria três containers leves rodando sobre um único SO hospedeiro, otimizando o uso de recursos e facilitando a gestão.

Made with GAMMA

FERRAMENTA ESSENCIAL

Docker: A Plataforma de Containers Mais Popular

Docker revolucionou a forma como desenvolvemos, empacotamos e executamos aplicações. Ele fornece uma plataforma completa para gerenciar o ciclo de vida dos containers, tornando-os acessíveis e práticos para equipes de todos os tamanhos.

Permite criar, distribuir e executar containers facilmente

Sua CLI intuitiva e ecossistema robusto simplificam tarefas complexas.

Dockerfile: receita para construir imagens consistentes

Garante que o ambiente da aplicação seja reproduzível em qualquer lugar.

Docker Compose: orquestração simples

Facilita a definição e execução de aplicações multi-container (ex: app + banco de dados + cache).

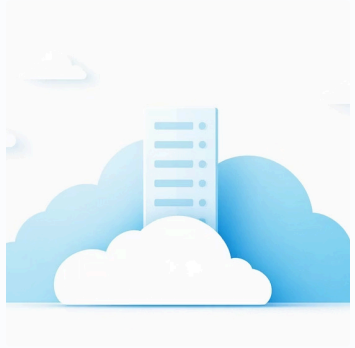
> Os benefícios do Docker são vastos: portabilidade entre ambientes, isolamento robusto para evitar conflitos, rapidez na inicialização e deploy, e consistência garantida do desenvolvimento à produção. É a base para um fluxo de trabalho DevOps eficiente.

Made with GAMMA

ESCALA E GESTÃO

Kubernetes: Orquestração Avançada de Containers

Quando o número de containers cresce e a complexidade do ambiente aumenta, Kubernetes (K8s) se torna indispensável. Ele é um sistema de código aberto para automatizar o deploy, escalonamento e gerenciamento de aplicações containerizadas.



Gerencia clusters de containers em escala

Automatiza o deploy, escalonamento e manutenção de milhares de containers.

Ideal para ambientes de produção

Proporciona alta disponibilidade, resiliência e auto-recuperação de falhas.

Conceitos-chave

Pods, Serviços, Deployments, Namespaces - elementos fundamentais para organizar e gerenciar suas aplicações.

Por exemplo, Kubernetes pode rodar automaticamente múltiplas réplicas de uma aplicação web, distribuindo o tráfego de entrada entre elas (balanceamento de carga) e garantindo que, mesmo que um nó falhe, a aplicação continue disponível.

Made with GAMMA

DEVOPS NA PRÁTICA

Por que usar Containers e Orquestração em DevOps?

A adoção de containers e orquestradores como Docker e Kubernetes é um divisor de águas para equipes DevOps. Essas tecnologias atuam como facilitadores para práticas e princípios que visam acelerar o desenvolvimento, garantir a confiabilidade e otimizar a operação de software.

01

Integração e Entrega Contínua (CI/CD) facilitadas

Containers padronizam os ambientes, eliminando o "funciona na minha máquina" e agilizando os pipelines de CI/CD.

02

Ambientes idênticos do desenvolvimento à produção

A imutabilidade das imagens de container garante que o que foi testado no desenvolvimento é exatamente o que será executado em produção.

03

Escalabilidade e gerenciamento simplificados

Orquestradores permitem escalar aplicações vertical e horizontalmente de forma automatizada, respondendo à demanda em tempo real.

04

Redução de custos e tempo de deploy

A eficiência no uso de recursos e a automação do deploy resultam em economia de infraestrutura e ciclos de entrega mais rápidos.

Made with GAMMA

MÃO NA MASSA

Desafio 2: Trabalho em Equipe com Docker ou Kubernetes

Este desafio prático é a sua oportunidade de solidificar o conhecimento sobre containers e orquestração. Trabalhem em equipe para construir uma aplicação simples, utilizando as ferramentas e conceitos apresentados.

Criar uma aplicação simples

Pode ser um "Hello World" web, uma API simples, ou qualquer serviço pequeno, containerizando-o com Docker ou deployando-o no Kubernetes.

Apresentar a arquitetura da solução

Expliquem como a aplicação foi estruturada, quais componentes foram utilizados (ex: Dockerfile, docker-compose.yml, YAMLs do Kubernetes).

Demonstrar comandos usados e benefícios

Mostrem os principais comandos (`docker run`, `kubectl apply`, `kubectl get pods`, etc.) e discorram sobre os benefícios práticos observados (portabilidade, escalabilidade, etc.).

Entregar apresentação clara e objetiva

Apresentem para a turma de forma concisa e didática, focando nos aprendizados e desafios superados. Usem esta apresentação GML como inspiração!

Made with GAMMA

SUCESSO NO DESAFIO

Dicas para o Desafio

Para garantir o sucesso no desafio e maximizar o aprendizado em equipe, considere estas dicas:

- **Dividir tarefas de forma estratégica:** Atribuam responsabilidades claras para cada membro da equipe, como a criação do Dockerfile, a configuração do docker-compose.yml (ou YAMLs do Kubernetes), a escrita dos scripts de deploy, e a documentação do projeto. Isso otimiza o tempo e permite que todos aprendam diferentes aspectos.
- **Testar localmente antes da apresentação:** Garanta que a aplicação e os comandos funcionem perfeitamente no ambiente local de cada membro antes de considerá-los prontos para a demonstração. Isso evita imprevistos e garante uma apresentação fluida.
- **Explorar imagens oficiais para acelerar o desenvolvimento:** O Docker Hub e outros repositórios contêm uma vasta gama de imagens base oficiais (Node.js, Python, Nginx, etc.). Utilizá-las pode poupar muito tempo na configuração inicial do ambiente da sua aplicação.
- **Usar comandos básicos para mostrar containers em execução e logs:** Prepare-se para demonstrar comandos como `docker ps` (para listar containers em execução), `docker logs [container_id]` (para ver a saída da aplicação), `kubectl get pods` (para verificar o status dos pods no Kubernetes), e `kubectl logs [pod_name]`. Isso valida o funcionamento da sua solução.
- **Documentar o processo:** Um bom README no repositório do projeto, explicando como configurar e rodar a aplicação, é crucial para a colaboração e para a avaliação.

Made with GAMMA

OLHANDO PARA FRENTE

Conclusão: O Futuro do DevOps com Containers

A jornada pelo mundo das Máquinas Virtuais e Containers, culminando neste desafio em equipe, é um passo crucial para compreender e atuar no cenário de tecnologia atual e futuro. As tecnologias de containerização e orquestração não são meras tendências, mas pilares fundamentais da modernização de TI.

1 Containers e orquestração são pilares da modernização de TI

Eles habilitam arquiteturas de microsserviços, nuvem nativa e promovem a agilidade necessária para o desenvolvimento de software moderno.

2 Dominar Docker e Kubernetes abre portas para carreiras em DevOps e Cloud

Profissionais com estas habilidades são altamente valorizados no mercado, sendo essenciais para a transformação digital das empresas.

3 O desafio é a oportunidade de aplicar teoria na prática e aprender colaborativamente

A experiência prática e o trabalho em equipe são inestimáveis para solidificar o conhecimento e desenvolver habilidades essenciais.

4 Vamos juntos construir ambientes mais ágeis, escaláveis e confiáveis!

Continuem explorando, questionando e inovando. O futuro do DevOps está em suas mãos.

Made with GAMMA